

A few myths

1. Malware is the reason for a slow computer.
2. Clearing the personal data from our hard drive will boost its performance.
3. Replacing and upgrading different components ensures guaranteed speed.
4. Cleaning the registry speeds up the system.
5. A fresh install of the operating system is the ultimate way to gain speed.

open source tools. Individuals who have faith in that tool for difficult computing tasks can really be certain that such open source tools won't disappear if their original creators stop working on them. They will always be supported by the open source community.

5. **Security:** Open source productivity tools are comparatively safer and steadier than the licensed ones. Just as anyone can view and change the code for different open source tools, they can also point out and correct the faults that a program's original developer might have missed out. Also, since many developers can work on different parts of open source tool code without actually asking for the approval of the original developer, they can

modify, correct and improve the code more quickly than in the case of licensed ones.

6. **Scope for continuous change and improvement:** There is always scope for change and improvement with open source productivity tools, as everyone can access the code and apply their innovation to it. There is a large community working for this, so the probability and rate of improvement is really high, compared to closed source software tools. 

References

- [1] <http://www.wikipedia.org/>
- [2] <http://www.makeuseof.com/>
- [3] <https://msdn.microsoft.com/>
- [4] <http://www.techrepublic.com/>

By: Vivek Ratan

The author is a B.Tech in electronics and instrumentation engineering. He is currently working as an automation test engineer at Infosys, Pune and as a freelance educator at LearnerKul, Pune. He can be reached at ratanvivek14@gmail.com for any suggestions or queries.

Continued from page 91...

Open source support for deep learning

TensorFlow is an open source software library released in 2015 by Google to make it easier for developers to design, build and train deep learning models. Actually, TensorFlow originated as an internal library that Google developers used to build models in-house. Multiple functionalities were added later, as it became a powerful tool.

At a higher level, TensorFlow is a Python library that allows users to express arbitrary computation as a graph of data flows. Nodes in this graph represent mathematical operations, whereas edges represent data that is communicated from one node to another. Data in TensorFlow is represented as tensors, which are nothing but multi-dimensional arrays (from 1D tensor, 2D tensor, 3D tensor... etc.). The TensorFlow library can be downloaded straight away from <https://www.tensorflow.org/>.

There are many other popular open source libraries that have popped up over the last few years for building neural networks. These are:

- 1) Theano: Built by the LISA lab at the University of Montreal (<https://pypi.python.org/pypi/Theano>)
- 2) Keras (<https://pypi.python.org/pypi/Keras>)
- 3) Torch: Largely maintained by the Facebook AI research team (<http://torch.ch/>)

All these options boast of a sizeable developer community, but of all these TensorFlow stands out because of its features such as the shorter data training time, ease of

writing and debugging code, availability of many classes of models, and multiple GPU support on a single machine. Also, it is well suited for production-ready use cases.

Deep learning has become an extremely active area of research and is finding many applications in real life challenges. Companies such as Google, Microsoft and Facebook are actively forming deep learning teams in-house. The coming days will witness more advances in this field—agents capable of mastering Atari games, driving cars, trading stocks profitably and controlling robots. 

References

- [1] Neuron: <http://science.howstuffworks.com/life/inside-the-mind/human-brain/brain1.htm>
- [2] <https://www.tensorflow.org/>
- [3] Gradient Descent: <https://iamtrask.github.io/2015/07/27/python-network-part2/>
- [4] <http://ruder.io/optimizing-gradient-descent/>
- [5] https://en.wikipedia.org/wiki/Geoffrey_Hinton
- [6] https://en.wikipedia.org/wiki/Yann_Lecun

By: Shashidhar Soppin

The author is a senior architect and has 16+ years of experience in the IT industry having specialised in virtualisation, cloud computing, Docker, open source, etc. He can be contacted at shashi.soppin@gmail.com.